

Tech Dive: Jupyter at LSST

Adam Thornton

Overview

- LSST: What and Why
- Architecture: Kubernetes + JupyterHub + JupyterLab
- Specific Implementation Challenges and Solutions

LSST

Feeds and Speeds

This is the usual overview of what the LSST is going to be doing, and how much of it there is.

Notebook Environment (AKA “nublado”)

This is the LSST Science Platform Interactive Notebook Component. Basically, it’s a way of letting scientists quickly iterate through hypotheses looking for the ones interesting enough to burn a lot of resources investigating.

My talk at JupyterCon 2018 (slides) is not a bad overview, if I do say so myself.

Architectural assumptions

Kubernetes is the right level of abstraction

You’re free to argue with me about this.

If you do, you’re wrong.

Containerization

Abstraction that lets you care about the application software rather than the lower layers.

We're using Docker.

Composability

Kubernetes abstractions (e.g. the service) are designed such that we can load-balance and (in some cases) get HA without having to work very hard at it. The deployment manages container lifecycles so we have the right number of a given component running. We don't have to manage the (miserable) Docker-container-port-to-host-port mapping stuff ourselves.

This is where Kubernetes is magnificent.

Ubiquity

- If you are not a data center service provider
Demand your service provider give you a Kubernetes interface. The major public clouds already do.
- If you are a data center service provider
You either already do provide a managed Kubernetes service or you're going to have to. The longer you wait the more it will hurt.
Again, you can argue with me. Again, you're wrong.

Orchestrateable

Kustomize, Terraform, Helm, or roll-your own. Each of the first three has advantages.

JupyterHub

Why write your own spawner? I haven't heard a convincing reason.

JupyterLab

No sense in starting, several years from Science First Light, with something that's already being supplanted.

You can still get the Classic Notebook view from it, if you have users with notebooks that rely on things JupyterLab doesn't have extensions for. Encourage them to write those extensions or at least open issues.

Implementation Challenges and Solutions

1. Authentication
2. Resource Control
3. Configuration
4. User Environments

Authentication

Make it someone else's problem

It's full of corner cases and harder than it looks.

Are you **really** such a special snowflake that "users are members of groups, and groups map to capabilities" won't work for you?

OAuth2 is nice

Wide support, good JupyterHub support, easy to add new providers.

This is our configuration.

SSO

Custom header checking/injection in an Nginx ingress with a diversion through OAuth2 flow, followed by passing around JWT.

Our ingress annotations and header validation and parsing.

Note that Node.js has a default maximum header size of 8KB.

Better SSO

CILogon+NCSA IDP supports association of identities, which is a nice feature. See if your OAuth2 provider can do it.

For instance, I'm usually signed into GitHub within ten minutes of logging on somewhere.

Resource Control

Group Membership

A group is really a mapping to a set of capabilities.

Any reasonable authentication provider should be able to also do multiple group memberships for an identity.

Capabilities are equivalent to resource entitlement

What a user is allowed to do is the union of the capabilities of each of their groups.

Namespace a user's resources in Kubernetes

- Quotas
CPU, RAM, and object count.
Construct different quotas for different groups.
- Ease of cleanup
Once you start constructing complex user environments, it's easy to leak.
Namespace teardown removes all namespaced resources; in our experience, everything but PVs.

Time is a resource

If you have a complex set of analysis tools, your images may be very large. Ours are 16GB now.

This can take a very long time to pull.

- Prepuller
Run something to continually pull some set of versions of your standard images. Couple with a CI system and by the time people show up in the morning, the new image is pulled.
Cuts startup time from 10 minutes to 15 seconds for us.
- Build around your stack
Don't take a base JupyterLab and add your software to it if your software is large.
Instead, add JupyterLab to your software.

Intermediate-scale parallelism

- Things too big to fit in a single Python process/cell
Say, a handful of columns across a couple billion rows. (GAIA DR2, "l" and "b" columns only)
- But not so big you want to go with full-on HTCondor yet
LSST DR11 final catalog size: 15PB.

- We use Dask
By the end of the survey, much that we would now use a batch environment for will be reasonable in an interactive Dask-like framework. 15PB of catalog data?
- Considerations for using Dask
 - Keeping Python libraries and versions synced
Use the same container with a different environmental flag set to say “be a Dask worker, not a JupyterLab server.”
In our environment, both Jupyter machinery and Dask machinery are small compared to our analysis software.
 - Need additional Role/ServiceAccount/Rolebinding to allow Lab to spawn Dask
We populate a Dask worker yml document at each login that does the right thing. Modify at your own risk and you’re still subject to quotas.
We anticipate very few users will ever need this level of control.
 - Resource limits can cause worker nodes to get reaped
Some attention to partitioning is still required.
- Now the user Lab container has to create other containers
But in the same namespace, so quotas are still easy.
- RBAC
It’s not that scary.
This is an example for JupyterHub.

Configuration

Modularity with ConfigMaps

This is a JupyterHub minimal configuration wrapper that loads the (sorted) contents of a configuration directory.

This is one of the files it loads.

Make your ConfigMaps generic.

Instance-specific values

Put them in templated environment, or in Secrets for sensitive data.

Don't be afraid to subclass right in your ConfigMaps

User Environments

Use a spawner options form to present choices

- Images
- Container sizes
- Mounted filesystems

You can use groups to control what's displayed.

Be the User

Pass information into the user container and do user setup as a semiprivileged user with tightly controlled sudo.

Then start the JupyterLab server as the user, in the user's home directory.

Do not give any sudo privileges to the user.

- Complex environmental variables

Set up gid/groupname mappings, uid/username, and parse in the shell on the far end...

This is what we've been doing, and we've found we need to...

- base64-encode the really complicated stuff

Here is how we do our initial Dask container template setup.

This gets silly fast. Instead try:

- ConfigMaps

Define ConfigMaps (which are namespaced) at spawn time and map them into the user's Lab container as read-only files.

Persistent Storage

You just need a consistent and persistent way to assign uids/gids.

Your LDAP system should already do this. GitHub has unique 32-bit identifiers for users and groups. Google will require you to map 64-bit IDs to 32-bit.

- Access Control is now a solved problem

You can use POSIX ACLs if there's something good old file permissions can't handle.

- NFS
 - Works, ubiquitous, but...
 - Performance
 - Locking
 - The use of non-default NFS options in Kubernetes requires hacky workarounds
- HostPath
 - “Get out of jail free.”
 - Jails exist for reasons.
 - Not officially supported for ReadWriteMany.
 - GPFS seems to work for us, with good performance, but YMMV.

Links

This Talk

This talk (source): <https://github.com/lsst-sqre/Jupyter-PCW-2019.git>.

This talk: <https://athornton.github.io/Jupyter-PCW-2019>.

Useful Repositories

The Notebook Aspect of the LSST Science Platform: <https://github.com/lsst-sqre/nublado.git> (example sources).

LSST JupyterHub Utilities: <https://github.com/lsst-sqre/jupyterhubutils> (prepuller and reposcanner).

LSST JupyterLab Utilities: <https://github.com/lsst-sqre/jupyterlabutils> (Dask cluster proxy for use in Kubernetes).

Questions

This talk: <https://athornton.github.io/Jupyter-PCW-2019>. (PDF)
Adam Thornton <athornton@lsst.org>